

CS-341L - Fall 2025
Cache Lab: Understanding Cache Memories
Part B - comparing human and AI generated code
This portion done in teams of two

Assigned: November 13, 2025

Due date for Part B: December 8, 2025 at 11:59 pm

1 Purpose

The purpose of this second part of the cachelab is to explore how AI performs in *generating optimized matrix-transpose code* for Cache Lab tasks compared to your own human-written code. You will learn, with an example, how to use AI responsibly, measure strengths/weaknesses, and communicate conclusions.

2 Logistics

This is a team project. You need to find a partner to work with from the students registered in the class.

3 Overview

Parts A and B of this lab will help you understand the impact that cache memories can have on the performance of your C programs.

3.1 Part B: Optimizing Matrix Transpose

This section contains first, the general description of this optimization component of the cache lab. We are copying it here so that you have everything you need in this statement (I hope), but you can always look back at the first statment for the cachelab, in case we missed something from there.

What we are changing in this occasion is that for each matrix size you need to write the new transpose function that will optimize the number of misses (i.e. have as few as possible) written by you or your partner or both without using any tools or any external sources; this will be the **human-generated code for the optimized transpose function (trans-human.c)**. Then you want to use a *genAI tool* to produce another

optimized transpose function which will be the **AI-generated optimized transpose function (ai-trans.c)**. Then you must execute both functions for each size and compare both the code and the performance of each.

3.2 General description of the optimization of Matrix transpose

In Part B you will write a transpose function in `trans.c` that causes as few cache misses as possible.

Let A denote a matrix, and A_{ij} denote the component on the i th row and j th column. The *transpose* of A , denoted A^T , is a matrix such that $A_{ij} = A_{ji}^T$.

To help you get started, we have given you an example transpose function in `trans.c` that computes the transpose of $N \times M$ matrix A and stores the results in $M \times N$ matrix B :

```
char trans_desc[] = "Simple row-wise scan transpose";
void trans(int M, int N, int A[N][M], int B[M][N])
```

The example transpose function is correct, but it is inefficient because the access pattern results in relatively many cache misses.

Your job in Part B is to write a similar function, called `transpose_submit`, that minimizes the number of cache misses across different sized matrices:

```
char transpose_submit_desc[] = "Transpose submission";
void transpose_submit(int M, int N, int A[N][M], int B[M][N]);
```

Do *not* change the description string ("Transpose submission") for your `transpose_submit` function. The autograder searches for this string to determine which transpose function to evaluate for credit.

You may use the provided simulator to solve the cache optimization problems in this part.

3.3 Programming Rules for Part B

- Include name and netID of the members of the team in the header comment for all versions of the optimized `trans.c` functions that you write and also the ones generated with AI tools, the name of the file will tell if it was human or AI generated, you may add the AI tool used and its version, after the names of the team members.
- Your code in all the `trans.c` must compile without warnings.
- You are allowed to define at most 12 local variables of type `int` per transpose function.¹
- You are not allowed to side-step the previous rule by using any variables of type `long` or by using any bit tricks to store more than one value to a single variable.

¹The reason for this restriction is that our testing code is not able to count references to the stack. We want you to limit your references to the stack and focus on the access patterns of the source and destination arrays.

- Your transpose functions may not use recursion.
- If you choose to use helper functions, you may not have more than 12 local variables on the stack at a time between your helper functions and your top level transpose function. For example, if your transpose declares 8 variables, and then you call a function which uses 4 variables, which calls another function which uses 2, you will have 14 variables on the stack, and you will be in violation of the rule.
- Your transpose functions may not modify array A. You may, however, do whatever you want with the contents of array B.
- You are NOT allowed to define any arrays in your code or to use any variant of `malloc`.

4 The new tasks for the Matrix Transpose Optimization

The list that follows shows the tasks you must perform to complete this assignment.

1. You and your partner will write one optimized matrix-transpose function for each matrix size “by hand” without any external tools; these are the `trans_human.c` solutions included all in one single `.c` file. (See section 6.1 for more details.)
 - (a) Record your code and performance of each of the `trans_human.c` solution.
 - (b) Store your own written code in `trans_human.c`
 - (c) Store you performance in `trans_human_results.csv`
2. Generate solutions to the same problem using a genAI tool.
 - (a) Record prompts and the entire conversation. Ensure you record the model information as well (i.e. the chatbot and version of it that you are using).
 - (b) Record EVERYTHING:
 - Store your AI’s written code in `ai_trans.c`
 - Store performances in `ai_trans_results.csv`
 - Store prompt log in a folder and make it organized by problem (matrix size).
3. Run performance tests and provide test metrics results in assignment tar file.
4. Record the quality of the generated code
 - Discuss readability, naming, structure, algorithmic strategy, look/feel, etc.
5. Discuss the results from AI and Human written code using the following guidance:
 - Discuss the performance and quality of the code.
 - Note patterns, compare AI and Human written code. Discuss pattern of how prompting differences affect AI performance, and strength and weakness of AI/Human generated code, etc.

- Pour out any idea you like just ensure that you have evidence to support that. You can't just say that AI is better at generating better commented code unless you have evidence from the result section to suggest this.
- The claims that you make must be testable.

6. Draw conclusions

- Analyze the results and summarize key ideas for the previous section.
- Create key conclusions or takeaways (in the form of a bullet list of sentences) from this test.
- Write a report that includes all the information above.

4.1 Report structure (Example)

You must produce a report with all the information described in section 4

1. Title and Summary – short summary and main claim(s).
2. Introduction and Motivation – why these matters; relation to Cache Lab learning goals.
3. Methods – the problems, model used, prompting method.
4. Data Collection – the performance and code quality
5. Exploratory Analysis/Discussion – explore possible reasons behind the results, discuss results and reasoning for possible ideas.
6. Conclusion — what you learned, conclusions for using AI in code generation.
7. References – a list of all the sources used to perform the tasks for this assignment. Include all the kinds of help used if any.
8. Appendices – If any, these may include: c code, log of prompts, intermediate results, etc.

Special restriction: please do not include screenshots in your report, they occupy a lot of space, make the pdf unmanageable, and are usually unreadable. The history of your interaction with the AI tools must be stored in plain text files with the .txt extension. The code must be stored in .c files.

4.2 AI Tool Policy

Your use of genAI tools for this assignment is subject to the following constraints:

- You may NOT use AI for your human-written code, it will violate the rules of the assignment and will get you a zero.
- For AI-generated code:
 - You may approach the AI however you like but must record protocol for each problem (the prompting style)
 - You MUST record at a minimum:
 - * Model name & version
 - * Exact prompts and follow-ups
 - * How the code evolved (story, not just the final answer)

5 What to submit

You will compress the following files into a tar file and submit the tar file on canvas at the place for this assignment:

Human generated code The `trans_human.c` file and the performance evaluation of this in the `trans_human_results.csv` file. You need to follow the instructions to evaluate the performance of this code.

AI generated code The best `ai_trans.c` file generated with AI tools and the `ai_trans_results.csv` with the performance evaluation of this code.

Report The report produced from the execution of the tasks described in section 4, with the contents described in that same section and in section 4.1.

6 Rubric for evaluating Part B

+10 pts: The submission is complete and well organized according to the specifications given in this document.

+ 26 pts: The performance evaluation of your human-generated code.

+ 24 pts: The quality of your report according to the following criteria: organization, clarity, presentation of the results, completeness.

+ 40 pts: The quality of your analysis of results and conclusions.

6.1 Performance Evaluation

Human and AI generated code for the optimized transpose function will be evaluated by you according to the following aspects.

The correctness and performance of your `transpose_submit` function on three different-sized output matrices:

- 32×32 ($M = 32, N = 32$)
- 64×64 ($M = 64, N = 64$)
- 61×67 ($M = 61, N = 67$)

For each matrix size, the performance of your `transpose_submit` function is evaluated by using `valgrind` to extract the address trace for your function, and then using the reference simulator to replay this trace on a cache with parameters ($s = 5, E = 1, b = 5$).

Your performance score for each matrix size scales linearly with the number of misses, m , up to some threshold:

- 32×32 : 8 points if $m < 300$, 0 points if $m > 600$
- 64×64 : 8 points if $m < 1,300$, 0 points if $m > 2,000$
- 61×67 : 10 points if $m < 2,000$, 0 points if $m > 3,000$

All code submitted must be correct to receive any performance points for a particular size. All code submitted only needs to be correct for these three cases and you can optimize it specifically for these three cases. In particular, it is perfectly OK for your function to explicitly check for the input sizes and implement separate code optimized for each case. It is probably easier to include the code for each matrix size in a single `.c` file for the human generated code and another single file for the ai-generated code.

7 Working on Part B

We have provided you with an autograding program, called `test-trans.c`, that tests the correctness and performance of each of the transpose functions that you have registered with the autograder.

You can register up to 100 versions of the transpose function in your `trans.c` file. Each transpose version has the following form:

```
/* Header comment */
char trans_simple_desc[] = "A simple transpose";
void trans_simple(int M, int N, int A[N][M], int B[M][N])
{
    /* your transpose code here */
}
```

Register a particular transpose function with the autograder by making a call of the form:

```
registerTransFunction(trans_simple, trans_simple_desc);
```

in the `registerFunctions` routine in `trans.c`. At runtime, the autograder will evaluate each registered transpose function and print the results. Of course, one of the registered functions must be the `transpose_submit` function that you are submitting for credit:

```
registerTransFunction(transpose_submit, transpose_submit_desc);
```

See the default `trans.c` function for an example of how this works.

The autograder takes the matrix size as input. It uses `valgrind` to generate a trace of each registered transpose function. It then evaluates each trace by running the reference simulator on a cache with parameters ($s = 5$, $E = 1$, $b = 5$).

For example, to test your registered transpose functions on a 32×32 matrix, rebuild `test-trans`, and then run it with the appropriate values for M and N :

```
linux> make
linux> ./test-trans -M 32 -N 32
Step 1: Evaluating registered transpose funcs for correctness:
func 0 (Transpose submission): correctness: 1
func 1 (Simple row-wise scan transpose): correctness: 1
func 2 (column-wise scan transpose): correctness: 1
func 3 (using a zig-zag access pattern): correctness: 1

Step 2: Generating memory traces for registered transpose funcs.

Step 3: Evaluating performance of registered transpose funcs (s=5, E=1, b=5)
func 0 (Transpose submission): hits:1766, misses:287, evictions:255
func 1 (Simple row-wise scan transpose): hits:870, misses:1183, evictions:1151
func 2 (column-wise scan transpose): hits:870, misses:1183, evictions:1151
func 3 (using a zig-zag access pattern): hits:1076, misses:977, evictions:945

Summary for official submission (func 0): correctness=1 misses=287
```

In this example, we have registered four different transpose functions in `trans.c`. The `test-trans` program tests each of the registered functions, displays the results for each, and extracts the results for the official submission.

Here are some hints and suggestions for working on Part B.

- The `test-trans` program saves the trace for function i in file `trace.fi`.² These trace files are invaluable debugging tools that can help you understand exactly where the hits and misses for each transpose function are coming from. To debug a particular function, simply run its trace through the reference simulator with the verbose option:

²Because `valgrind` introduces many stack accesses that have nothing to do with your code, we have filtered out all stack accesses from the trace. This is why we have banned local arrays and placed limits on the number of local variables.

```
linux> ./csim-ref -v -s 5 -E 1 -b 5 -t trace.f0
S 68312c,1 miss
L 683140,8 miss
L 683124,4 hit
L 683120,4 hit
L 603124,4 miss eviction
S 6431a0,4 miss
...
```

- Since your transpose function is being evaluated on a direct-mapped cache, conflict misses are a potential problem. Think about the potential for conflict misses in your code, especially along the diagonal. Try to think of access patterns that will decrease the number of these conflict misses.
- Blocking is a useful technique for reducing cache misses. See the following url

<http://csapp.cs.cmu.edu/public/waside/waside-blocking.pdf>

for more information.

7.1 Putting it all Together

We have provided you with a *driver program*, called `./driver.py`, that performs a complete evaluation of your simulator and transpose code. This is the same program your instructor uses to evaluate your handins. The driver uses `test-csim` to evaluate your simulator, and it uses `test-trans` to evaluate your submitted transpose function on the three matrix sizes. Then it prints a summary of your results and the points you have earned.

To run the driver, type:

```
linux> ./driver.py
```

Each time you type `make` in the `cachelab-handout` directory, the Makefile creates a tarball, called `userid-handin.tar`, that contains your current `csim.c` and `trans.c` files.

IMPORTANT: Do not create the handin tarball on a Windows or Mac machine, and do not handin files in any other archive format, such as `.zip`, `.gzip`, or `.tgz` files.